

Booth Multiplier DataFlow

Prompt Strategy: Zero Shot

Prompt:

Generate synthesizable Verilog code for a 16-bit signed Booth Multiplier.

Requirements:

- Implement Radix-2 Booth multiplication.
- Use Dataflow RTL style only.
- Target FPGA: Xilinx Artix-7 xc7a100tcsg324-2.
- Use continuous assignments and combinational expressions.
- Implement Booth encoding and partial-product generation.
- Do not use always, always_comb, or always_ff.
- Do not use behavioral procedural blocks.
- Inputs:
 - input signed [15:0] multiplicand
 - input signed [15:0] multiplier
- Output:
 - output signed [31:0] product
- Generate synthesizable Verilog only.
- Output code only.

Code:

```
module booth_multiplier_16bit (
```

```
input signed [15:0] multiplicand,
input signed [15:0] multiplier,
output signed [31:0] product
);

wire [16:0] booth_bits;

assign booth_bits = {multiplier, 1'b0};

wire signed [31:0] pp0;
wire signed [31:0] pp1;
wire signed [31:0] pp2;
wire signed [31:0] pp3;
wire signed [31:0] pp4;
wire signed [31:0] pp5;
wire signed [31:0] pp6;
wire signed [31:0] pp7;
wire signed [31:0] pp8;
wire signed [31:0] pp9;
wire signed [31:0] pp10;
wire signed [31:0] pp11;
wire signed [31:0] pp12;
wire signed [31:0] pp13;
wire signed [31:0] pp14;
wire signed [31:0] pp15;
```

```
assign pp0 = (booth_bits[1:0] == 2'b01) ? ($signed(multiplicand) <<< 0) :  
    (booth_bits[1:0] == 2'b10) ? -($signed(multiplicand) <<< 0) :  
    32'sd0;
```

```
assign pp1 = (booth_bits[2:1] == 2'b01) ? ($signed(multiplicand) <<< 1) :  
    (booth_bits[2:1] == 2'b10) ? -($signed(multiplicand) <<< 1) :  
    32'sd0;
```

```
assign pp2 = (booth_bits[3:2] == 2'b01) ? ($signed(multiplicand) <<< 2) :  
    (booth_bits[3:2] == 2'b10) ? -($signed(multiplicand) <<< 2) :  
    32'sd0;
```

```
assign pp3 = (booth_bits[4:3] == 2'b01) ? ($signed(multiplicand) <<< 3) :  
    (booth_bits[4:3] == 2'b10) ? -($signed(multiplicand) <<< 3) :  
    32'sd0;
```

```
assign pp4 = (booth_bits[5:4] == 2'b01) ? ($signed(multiplicand) <<< 4) :  
    (booth_bits[5:4] == 2'b10) ? -($signed(multiplicand) <<< 4) :  
    32'sd0;
```

```
assign pp5 = (booth_bits[6:5] == 2'b01) ? ($signed(multiplicand) <<< 5) :  
    (booth_bits[6:5] == 2'b10) ? -($signed(multiplicand) <<< 5) :  
    32'sd0;
```

```
assign pp6 = (booth_bits[7:6] == 2'b01) ? ($signed(multiplicand) <<< 6) :  
    (booth_bits[7:6] == 2'b10) ? -($signed(multiplicand) <<< 6) :
```

```
32'sd0;
```

```
assign pp7 = (booth_bits[8:7] == 2'b01) ? ($signed(multiplicand) <<< 7) :
```

```
(booth_bits[8:7] == 2'b10) ? -($signed(multiplicand) <<< 7) :
```

```
32'sd0;
```

```
assign pp8 = (booth_bits[9:8] == 2'b01) ? ($signed(multiplicand) <<< 8) :
```

```
(booth_bits[9:8] == 2'b10) ? -($signed(multiplicand) <<< 8) :
```

```
32'sd0;
```

```
assign pp9 = (booth_bits[10:9] == 2'b01) ? ($signed(multiplicand) <<< 9) :
```

```
(booth_bits[10:9] == 2'b10) ? -($signed(multiplicand) <<< 9) :
```

```
32'sd0;
```

```
assign pp10 = (booth_bits[11:10] == 2'b01) ? ($signed(multiplicand) <<< 10) :
```

```
(booth_bits[11:10] == 2'b10) ? -($signed(multiplicand) <<< 10) :
```

```
32'sd0;
```

```
assign pp11 = (booth_bits[12:11] == 2'b01) ? ($signed(multiplicand) <<< 11) :
```

```
(booth_bits[12:11] == 2'b10) ? -($signed(multiplicand) <<< 11) :
```

```
32'sd0;
```

```
assign pp12 = (booth_bits[13:12] == 2'b01) ? ($signed(multiplicand) <<< 12) :
```

```
(booth_bits[13:12] == 2'b10) ? -($signed(multiplicand) <<< 12) :
```

```
32'sd0;
```

```
assign pp13 = (booth_bits[14:13] == 2'b01) ? ($signed(multiplicand) <<< 13) :  
    (booth_bits[14:13] == 2'b10) ? -($signed(multiplicand) <<< 13) :  
    32'sd0;
```

```
assign pp14 = (booth_bits[15:14] == 2'b01) ? ($signed(multiplicand) <<< 14) :  
    (booth_bits[15:14] == 2'b10) ? -($signed(multiplicand) <<< 14) :  
    32'sd0;
```

```
assign pp15 = (booth_bits[16:15] == 2'b01) ? ($signed(multiplicand) <<< 15) :  
    (booth_bits[16:15] == 2'b10) ? -($signed(multiplicand) <<< 15) :  
    32'sd0;
```

```
assign product =  
    pp0 + pp1 + pp2 + pp3  
    + pp4 + pp5 + pp6 + pp7  
    + pp8 + pp9 + pp10 + pp11  
    + pp12 + pp13 + pp14 + pp15;
```

```
endmodule
```

Prompt Strategy: Few Shot

Prompt:

Example of Dataflow RTL coding style:

```
module adder(
```

```
input [7:0] a,  
input [7:0] b,  
output [8:0] sum  
);
```

```
assign sum = a + b;
```

```
endmodule
```

Using the same Dataflow RTL style shown above, generate synthesizable Verilog code for a 16-bit signed Booth Multiplier.

Requirements:

- Radix-2 Booth multiplication.
- Dataflow RTL only.
- Continuous assignments preferred.
- No procedural blocks.
- No always statements.
- Target FPGA: xc7a100tcsg324-2.
- Output code only.

Code:

```
module booth_multiplier_16bit (  
    input signed [15:0] multiplicand,  
    input signed [15:0] multiplier,  
    output signed [31:0] product
```

);

wire signed [31:0] M;

assign M = {{16{multiplicand[15]}}, multiplicand};

wire signed [31:0] pp0;

wire signed [31:0] pp1;

wire signed [31:0] pp2;

wire signed [31:0] pp3;

wire signed [31:0] pp4;

wire signed [31:0] pp5;

wire signed [31:0] pp6;

wire signed [31:0] pp7;

wire signed [31:0] pp8;

wire signed [31:0] pp9;

wire signed [31:0] pp10;

wire signed [31:0] pp11;

wire signed [31:0] pp12;

wire signed [31:0] pp13;

wire signed [31:0] pp14;

wire signed [31:0] pp15;

assign pp0 = ({multiplier[0], 1'b0} == 2'b01) ? (M <<< 0) :

({multiplier[0], 1'b0} == 2'b10) ? -(M <<< 0) :

32'sd0;

```
assign pp1 = ({multiplier[1], multiplier[0]} == 2'b01) ? (M <<< 1) :  
    ({multiplier[1], multiplier[0]} == 2'b10) ? -(M <<< 1) :  
    32'sd0;
```

```
assign pp2 = ({multiplier[2], multiplier[1]} == 2'b01) ? (M <<< 2) :  
    ({multiplier[2], multiplier[1]} == 2'b10) ? -(M <<< 2) :  
    32'sd0;
```

```
assign pp3 = ({multiplier[3], multiplier[2]} == 2'b01) ? (M <<< 3) :  
    ({multiplier[3], multiplier[2]} == 2'b10) ? -(M <<< 3) :  
    32'sd0;
```

```
assign pp4 = ({multiplier[4], multiplier[3]} == 2'b01) ? (M <<< 4) :  
    ({multiplier[4], multiplier[3]} == 2'b10) ? -(M <<< 4) :  
    32'sd0;
```

```
assign pp5 = ({multiplier[5], multiplier[4]} == 2'b01) ? (M <<< 5) :  
    ({multiplier[5], multiplier[4]} == 2'b10) ? -(M <<< 5) :  
    32'sd0;
```

```
assign pp6 = ({multiplier[6], multiplier[5]} == 2'b01) ? (M <<< 6) :  
    ({multiplier[6], multiplier[5]} == 2'b10) ? -(M <<< 6) :  
    32'sd0;
```

```
assign pp7 = ({multiplier[7], multiplier[6]} == 2'b01) ? (M <<< 7) :  
    ({multiplier[7], multiplier[6]} == 2'b10) ? -(M <<< 7) :
```


32'sd0;

assign pp8 = ({multiplier[8], multiplier[7]} == 2'b01) ? (M <<< 8) :

({multiplier[8], multiplier[7]} == 2'b10) ? -(M <<< 8) :

32'sd0;

assign pp9 = ({multiplier[9], multiplier[8]} == 2'b01) ? (M <<< 9) :

({multiplier[9], multiplier[8]} == 2'b10) ? -(M <<< 9) :

32'sd0;

assign pp10 = ({multiplier[10], multiplier[9]} == 2'b01) ? (M <<< 10) :

({multiplier[10], multiplier[9]} == 2'b10) ? -(M <<< 10) :

32'sd0;

assign pp11 = ({multiplier[11], multiplier[10]} == 2'b01) ? (M <<< 11) :

({multiplier[11], multiplier[10]} == 2'b10) ? -(M <<< 11) :

32'sd0;

assign pp12 = ({multiplier[12], multiplier[11]} == 2'b01) ? (M <<< 12) :

({multiplier[12], multiplier[11]} == 2'b10) ? -(M <<< 12) :

32'sd0;

assign pp13 = ({multiplier[13], multiplier[12]} == 2'b01) ? (M <<< 13) :

({multiplier[13], multiplier[12]} == 2'b10) ? -(M <<< 13) :

32'sd0;

```

assign pp14 = ({multiplier[14], multiplier[13]} == 2'b01) ? (M <<< 14) :
    ({multiplier[14], multiplier[13]} == 2'b10) ? -(M <<< 14) :
    32'sd0;

```

```

assign pp15 = ({multiplier[15], multiplier[14]} == 2'b01) ? (M <<< 15) :
    ({multiplier[15], multiplier[14]} == 2'b10) ? -(M <<< 15) :
    32'sd0;

```

```

assign product =
    pp0 + pp1 + pp2 + pp3
    + pp4 + pp5 + pp6 + pp7
    + pp8 + pp9 + pp10 + pp11
    + pp12 + pp13 + pp14 + pp15;

```

```

endmodule

```

Prompt Strategy: Chain of Thought

Prompt:

Design a 16-bit signed Booth Multiplier.

Before generating RTL:

1. Analyze the Booth multiplication algorithm.
2. Determine Booth encoding logic.
3. Determine partial-product generation.

4. Determine how to express the datapath using dataflow modeling.
5. Determine final summation structure.

After completing the reasoning process, generate synthesizable Dataflow Verilog.

Requirements:

- Radix-2 Booth multiplication.
- Dataflow RTL style only.
- No procedural blocks.
- No always statements.
- Target FPGA: xc7a100tcsg324-2.
- Output complete code only.

Code:

```
module booth_multiplier_16bit (  
    input signed [15:0] multiplicand,  
    input signed [15:0] multiplier,  
    output signed [31:0] product  
);  
  
wire signed [31:0] M = {{16{multiplicand[15]}}, multiplicand};  
  
wire q_m1 = 1'b0;  
  
wire signed [31:0] pp0 = ({multiplier[0], q_m1} == 2'b01) ? (M <<< 0) :
```

$(\{\text{multiplier}[0], q_m1\} == 2'b10) ? -(M \lll 0) : 32'sd0;$

wire signed [31:0] pp1 = $(\{\text{multiplier}[1], \text{multiplier}[0]\} == 2'b01) ? (M \lll 1) :$

$(\{\text{multiplier}[1], \text{multiplier}[0]\} == 2'b10) ? -(M \lll 1) : 32'sd0;$

wire signed [31:0] pp2 = $(\{\text{multiplier}[2], \text{multiplier}[1]\} == 2'b01) ? (M \lll 2) :$

$(\{\text{multiplier}[2], \text{multiplier}[1]\} == 2'b10) ? -(M \lll 2) : 32'sd0;$

wire signed [31:0] pp3 = $(\{\text{multiplier}[3], \text{multiplier}[2]\} == 2'b01) ? (M \lll 3) :$

$(\{\text{multiplier}[3], \text{multiplier}[2]\} == 2'b10) ? -(M \lll 3) : 32'sd0;$

wire signed [31:0] pp4 = $(\{\text{multiplier}[4], \text{multiplier}[3]\} == 2'b01) ? (M \lll 4) :$

$(\{\text{multiplier}[4], \text{multiplier}[3]\} == 2'b10) ? -(M \lll 4) : 32'sd0;$

wire signed [31:0] pp5 = $(\{\text{multiplier}[5], \text{multiplier}[4]\} == 2'b01) ? (M \lll 5) :$

$(\{\text{multiplier}[5], \text{multiplier}[4]\} == 2'b10) ? -(M \lll 5) : 32'sd0;$

wire signed [31:0] pp6 = $(\{\text{multiplier}[6], \text{multiplier}[5]\} == 2'b01) ? (M \lll 6) :$

$(\{\text{multiplier}[6], \text{multiplier}[5]\} == 2'b10) ? -(M \lll 6) : 32'sd0;$

wire signed [31:0] pp7 = $(\{\text{multiplier}[7], \text{multiplier}[6]\} == 2'b01) ? (M \lll 7) :$

$(\{\text{multiplier}[7], \text{multiplier}[6]\} == 2'b10) ? -(M \lll 7) : 32'sd0;$

wire signed [31:0] pp8 = $(\{\text{multiplier}[8], \text{multiplier}[7]\} == 2'b01) ? (M \lll 8) :$

$(\{\text{multiplier}[8], \text{multiplier}[7]\} == 2'b10) ? -(M \lll 8) : 32'sd0;$

```
wire signed [31:0] pp9 = ({multiplier[9], multiplier[8]} == 2'b01) ? (M <<< 9) :  
    ({multiplier[9], multiplier[8]} == 2'b10) ? -(M <<< 9) : 32'sd0;
```

```
wire signed [31:0] pp10 = ({multiplier[10], multiplier[9]} == 2'b01) ? (M <<< 10) :  
    ({multiplier[10], multiplier[9]} == 2'b10) ? -(M <<< 10) : 32'sd0;
```

```
wire signed [31:0] pp11 = ({multiplier[11], multiplier[10]} == 2'b01) ? (M <<< 11) :  
    ({multiplier[11], multiplier[10]} == 2'b10) ? -(M <<< 11) : 32'sd0;
```

```
wire signed [31:0] pp12 = ({multiplier[12], multiplier[11]} == 2'b01) ? (M <<< 12) :  
    ({multiplier[12], multiplier[11]} == 2'b10) ? -(M <<< 12) : 32'sd0;
```

```
wire signed [31:0] pp13 = ({multiplier[13], multiplier[12]} == 2'b01) ? (M <<< 13) :  
    ({multiplier[13], multiplier[12]} == 2'b10) ? -(M <<< 13) : 32'sd0;
```

```
wire signed [31:0] pp14 = ({multiplier[14], multiplier[13]} == 2'b01) ? (M <<< 14) :  
    ({multiplier[14], multiplier[13]} == 2'b10) ? -(M <<< 14) : 32'sd0;
```

```
wire signed [31:0] pp15 = ({multiplier[15], multiplier[14]} == 2'b01) ? (M <<< 15) :  
    ({multiplier[15], multiplier[14]} == 2'b10) ? -(M <<< 15) : 32'sd0;
```

```
assign product =
```

```
    pp0 + pp1 + pp2 + pp3  
    + pp4 + pp5 + pp6 + pp7  
    + pp8 + pp9 + pp10 + pp11  
    + pp12 + pp13 + pp14 + pp15;
```

endmodule

Prompt Strategy: Role Prompting

Prompt:

You are a Senior FPGA Design Engineer specializing in arithmetic datapaths and FPGA DSP architectures.

Generate production-quality synthesizable Verilog for a 16-bit signed Booth Multiplier.

Requirements:

- Radix-2 Booth multiplication.
- Dataflow RTL style only.
- Implement Booth encoding using combinational expressions.
- Implement partial-product generation using continuous assignments.
- Avoid procedural constructs.
- No always, always_comb, or always_ff.
- Optimized for Xilinx Artix-7 xc7a100tcsg324-2.
- Output code only.

Code:

```
module booth_multiplier_16 (  
    input signed [15:0] multiplicand,  
    input signed [15:0] multiplier,
```

```

output signed [31:0] product

);

// Extended multiplicand for sign-preserving shifts
wire signed [31:0] m_ext = {{16{multiplicand[15]}}, multiplicand};

// Booth recoding bits
wire [16:0] y = {multiplier, 1'b0};

// Partial products
wire signed [31:0] pp0 = ( y[1] & ~y[0] ) ? (m_ext <<< 0) :
    (~y[1] & y[0] ) ? -(m_ext <<< 0) :
    32'sd0;

wire signed [31:0] pp1 = ( y[2] & ~y[1] ) ? (m_ext <<< 1) :
    (~y[2] & y[1] ) ? -(m_ext <<< 1) :
    32'sd0;

wire signed [31:0] pp2 = ( y[3] & ~y[2] ) ? (m_ext <<< 2) :
    (~y[3] & y[2] ) ? -(m_ext <<< 2) :
    32'sd0;

wire signed [31:0] pp3 = ( y[4] & ~y[3] ) ? (m_ext <<< 3) :
    (~y[4] & y[3] ) ? -(m_ext <<< 3) :
    32'sd0;

```

```
wire signed [31:0] pp4 = ( y[5] & ~y[4] ) ? (m_ext <<< 4 ) :  
    (~y[5] & y[4] ) ? -(m_ext <<< 4 ) :  
    32'sd0;
```

```
wire signed [31:0] pp5 = ( y[6] & ~y[5] ) ? (m_ext <<< 5 ) :  
    (~y[6] & y[5] ) ? -(m_ext <<< 5 ) :  
    32'sd0;
```

```
wire signed [31:0] pp6 = ( y[7] & ~y[6] ) ? (m_ext <<< 6 ) :  
    (~y[7] & y[6] ) ? -(m_ext <<< 6 ) :  
    32'sd0;
```

```
wire signed [31:0] pp7 = ( y[8] & ~y[7] ) ? (m_ext <<< 7 ) :  
    (~y[8] & y[7] ) ? -(m_ext <<< 7 ) :  
    32'sd0;
```

```
wire signed [31:0] pp8 = ( y[9] & ~y[8] ) ? (m_ext <<< 8 ) :  
    (~y[9] & y[8] ) ? -(m_ext <<< 8 ) :  
    32'sd0;
```

```
wire signed [31:0] pp9 = ( y[10] & ~y[9] ) ? (m_ext <<< 9 ) :  
    (~y[10] & y[9] ) ? -(m_ext <<< 9 ) :  
    32'sd0;
```

```
wire signed [31:0] pp10 = ( y[11] & ~y[10] ) ? (m_ext <<< 10 ) :  
    (~y[11] & y[10] ) ? -(m_ext <<< 10 ) :
```



```
32'sd0;
```

```
wire signed [31:0] pp11 = ( y[12] & ~y[11]) ? (m_ext <<< 11) :
```

```
(~y[12] & y[11]) ? -(m_ext <<< 11) :
```

```
32'sd0;
```

```
wire signed [31:0] pp12 = ( y[13] & ~y[12]) ? (m_ext <<< 12) :
```

```
(~y[13] & y[12]) ? -(m_ext <<< 12) :
```

```
32'sd0;
```

```
wire signed [31:0] pp13 = ( y[14] & ~y[13]) ? (m_ext <<< 13) :
```

```
(~y[14] & y[13]) ? -(m_ext <<< 13) :
```

```
32'sd0;
```

```
wire signed [31:0] pp14 = ( y[15] & ~y[14]) ? (m_ext <<< 14) :
```

```
(~y[15] & y[14]) ? -(m_ext <<< 14) :
```

```
32'sd0;
```

```
wire signed [31:0] pp15 = ( y[16] & ~y[15]) ? (m_ext <<< 15) :
```

```
(~y[16] & y[15]) ? -(m_ext <<< 15) :
```

```
32'sd0;
```

```
// Final summation
```

```
assign product =
```

```
    pp0 + pp1 + pp2 + pp3
```

```
    + pp4 + pp5 + pp6 + pp7
```

+ pp8 + pp9 + pp10 + pp11
+ pp12 + pp13 + pp14 + pp15;

endmodule

Prompt Strategy: Instruction Format

Prompt:

TASK:

Generate a 16-bit signed Booth Multiplier.

ALGORITHM:

Radix-2 Booth Multiplication

LANGUAGE:

Verilog

RTL STYLE:

Dataflow

TARGET FPGA:

Xilinx Artix-7 xc7a100tcsg324-2

INPUTS:

input signed [15:0] multiplicand

input signed [15:0] multiplier

OUTPUT:

output signed [31:0] product

RULES:

- Use continuous assignments.
- Use combinational expressions.
- Implement Booth encoding.
- Implement partial-product generation.
- No procedural blocks.
- No always statements.

OUTPUT FORMAT:

Verilog code only.

Code:

```
module booth_multiplier_16 (  
    input signed [15:0] multiplicand,  
    input signed [15:0] multiplier,  
    output signed [31:0] product  
);  
  
    // Booth recoding bits  
    wire [16:0] booth_bits;  
    assign booth_bits = {multiplier, 1'b0};
```

```
// Sign-extended multiplicand
```

```
wire signed [31:0] M;
```

```
assign M = {{16{multiplicand[15]}}, multiplicand};
```

```
// Booth partial products
```

```
wire signed [31:0] pp0;
```

```
wire signed [31:0] pp1;
```

```
wire signed [31:0] pp2;
```

```
wire signed [31:0] pp3;
```

```
wire signed [31:0] pp4;
```

```
wire signed [31:0] pp5;
```

```
wire signed [31:0] pp6;
```

```
wire signed [31:0] pp7;
```

```
wire signed [31:0] pp8;
```

```
wire signed [31:0] pp9;
```

```
wire signed [31:0] pp10;
```

```
wire signed [31:0] pp11;
```

```
wire signed [31:0] pp12;
```

```
wire signed [31:0] pp13;
```

```
wire signed [31:0] pp14;
```

```
wire signed [31:0] pp15;
```

```
assign pp0 = (booth_bits[1:0] == 2'b01) ? ( M <<< 0 ) :
```

```
    (booth_bits[1:0] == 2'b10) ? (-M <<< 0 ) :
```

```
    32'sd0;
```

```
assign pp1 = (booth_bits[2:1] == 2'b01) ? ( M <<< 1 ) :  
    (booth_bits[2:1] == 2'b10) ? (-M <<< 1 ) :  
    32'sd0;
```

```
assign pp2 = (booth_bits[3:2] == 2'b01) ? ( M <<< 2 ) :  
    (booth_bits[3:2] == 2'b10) ? (-M <<< 2 ) :  
    32'sd0;
```

```
assign pp3 = (booth_bits[4:3] == 2'b01) ? ( M <<< 3 ) :  
    (booth_bits[4:3] == 2'b10) ? (-M <<< 3 ) :  
    32'sd0;
```

```
assign pp4 = (booth_bits[5:4] == 2'b01) ? ( M <<< 4 ) :  
    (booth_bits[5:4] == 2'b10) ? (-M <<< 4 ) :  
    32'sd0;
```

```
assign pp5 = (booth_bits[6:5] == 2'b01) ? ( M <<< 5 ) :  
    (booth_bits[6:5] == 2'b10) ? (-M <<< 5 ) :  
    32'sd0;
```

```
assign pp6 = (booth_bits[7:6] == 2'b01) ? ( M <<< 6 ) :  
    (booth_bits[7:6] == 2'b10) ? (-M <<< 6 ) :  
    32'sd0;
```

```
assign pp7 = (booth_bits[8:7] == 2'b01) ? ( M <<< 7 ) :
```

```
(booth_bits[8:7] == 2'b10) ? (-M <<< 7) :  
32'sd0;
```

```
assign pp8 = (booth_bits[9:8] == 2'b01) ? ( M <<< 8 ) :  
            (booth_bits[9:8] == 2'b10) ? (-M <<< 8 ) :  
            32'sd0;
```

```
assign pp9 = (booth_bits[10:9] == 2'b01) ? ( M <<< 9 ) :  
            (booth_bits[10:9] == 2'b10) ? (-M <<< 9 ) :  
            32'sd0;
```

```
assign pp10 = (booth_bits[11:10] == 2'b01) ? ( M <<< 10 ) :  
            (booth_bits[11:10] == 2'b10) ? (-M <<< 10 ) :  
            32'sd0;
```

```
assign pp11 = (booth_bits[12:11] == 2'b01) ? ( M <<< 11 ) :  
            (booth_bits[12:11] == 2'b10) ? (-M <<< 11 ) :  
            32'sd0;
```

```
assign pp12 = (booth_bits[13:12] == 2'b01) ? ( M <<< 12 ) :  
            (booth_bits[13:12] == 2'b10) ? (-M <<< 12 ) :  
            32'sd0;
```

```
assign pp13 = (booth_bits[14:13] == 2'b01) ? ( M <<< 13 ) :  
            (booth_bits[14:13] == 2'b10) ? (-M <<< 13 ) :  
            32'sd0;
```

```
assign pp14 = (booth_bits[15:14] == 2'b01) ? ( M <<< 14 ) :
```

```
(booth_bits[15:14] == 2'b10) ? (-M <<< 14 ) :
```

```
32'sd0;
```

```
assign pp15 = (booth_bits[16:15] == 2'b01) ? ( M <<< 15 ) :
```

```
(booth_bits[16:15] == 2'b10) ? (-M <<< 15 ) :
```

```
32'sd0;
```

```
// Final summation
```

```
assign product =
```

```
pp0 + pp1 + pp2 + pp3
```

```
+ pp4 + pp5 + pp6 + pp7
```

```
+ pp8 + pp9 + pp10 + pp11
```

```
+ pp12 + pp13 + pp14 + pp15;
```

```
endmodule
```

Prompt Strategy: Negative Prompting

Prompt:

Generate synthesizable Verilog for a 16-bit signed Booth Multiplier.

Requirements:

- Radix-2 Booth multiplication.
- Dataflow RTL only.

Do NOT:

- Use always blocks.
- Use always_comb.
- Use always_ff.
- Use procedural code.
- Use sequential logic.
- Use behavioral state machines.
- Use clocked processes.

Must:

- Use continuous assignments.
- Use combinational expressions.
- Implement Booth encoding.
- Implement partial-product generation.

Target FPGA:

xc7a100tcsg324-2

Output code only.

Code:

```
module booth_multiplier_16bit (  
    input signed [15:0] multiplicand,  
    input signed [15:0] multiplier,
```



```
    output signed [31:0] product  
);
```

```
wire [16:0] booth_bits;
```

```
assign booth_bits = {multiplier, 1'b0};
```

```
wire signed [31:0] pp0;
```

```
wire signed [31:0] pp1;
```

```
wire signed [31:0] pp2;
```

```
wire signed [31:0] pp3;
```

```
wire signed [31:0] pp4;
```

```
wire signed [31:0] pp5;
```

```
wire signed [31:0] pp6;
```

```
wire signed [31:0] pp7;
```

```
wire signed [31:0] pp8;
```

```
wire signed [31:0] pp9;
```

```
wire signed [31:0] pp10;
```

```
wire signed [31:0] pp11;
```

```
wire signed [31:0] pp12;
```

```
wire signed [31:0] pp13;
```

```
wire signed [31:0] pp14;
```

```
wire signed [31:0] pp15;
```

```
assign pp0 = (booth_bits[1:0] == 2'b01) ? ($signed(multiplicand) <<< 0) :  
    (booth_bits[1:0] == 2'b10) ? -($signed(multiplicand) <<< 0) : 32'sd0;
```

```

assign pp1 = (booth_bits[2:1] == 2'b01) ? ($signed(multiplicand) <<< 1) :
    (booth_bits[2:1] == 2'b10) ? -($signed(multiplicand) <<< 1) : 32'sd0;

assign pp2 = (booth_bits[3:2] == 2'b01) ? ($signed(multiplicand) <<< 2) :
    (booth_bits[3:2] == 2'b10) ? -($signed(multiplicand) <<< 2) : 32'sd0;

assign pp3 = (booth_bits[4:3] == 2'b01) ? ($signed(multiplicand) <<< 3) :
    (booth_bits[4:3] == 2'b10) ? -($signed(multiplicand) <<< 3) : 32'sd0;

assign pp4 = (booth_bits[5:4] == 2'b01) ? ($signed(multiplicand) <<< 4) :
    (booth_bits[5:4] == 2'b10) ? -($signed(multiplicand) <<< 4) : 32'sd0;

assign pp5 = (booth_bits[6:5] == 2'b01) ? ($signed(multiplicand) <<< 5) :
    (booth_bits[6:5] == 2'b10) ? -($signed(multiplicand) <<< 5) : 32'sd0;

assign pp6 = (booth_bits[7:6] == 2'b01) ? ($signed(multiplicand) <<< 6) :
    (booth_bits[7:6] == 2'b10) ? -($signed(multiplicand) <<< 6) : 32'sd0;

assign pp7 = (booth_bits[8:7] == 2'b01) ? ($signed(multiplicand) <<< 7) :
    (booth_bits[8:7] == 2'b10) ? -($signed(multiplicand) <<< 7) : 32'sd0;

assign pp8 = (booth_bits[9:8] == 2'b01) ? ($signed(multiplicand) <<< 8) :
    (booth_bits[9:8] == 2'b10) ? -($signed(multiplicand) <<< 8) : 32'sd0;

assign pp9 = (booth_bits[10:9] == 2'b01) ? ($signed(multiplicand) <<< 9) :

```

```
(booth_bits[10:9] == 2'b10) ? -($signed(multiplicand) <<< 9) : 32'sd0;
```

```
assign pp10 = (booth_bits[11:10] == 2'b01) ? ($signed(multiplicand) <<< 10) :
```

```
(booth_bits[11:10] == 2'b10) ? -($signed(multiplicand) <<< 10) : 32'sd0;
```

```
assign pp11 = (booth_bits[12:11] == 2'b01) ? ($signed(multiplicand) <<< 11) :
```

```
(booth_bits[12:11] == 2'b10) ? -($signed(multiplicand) <<< 11) : 32'sd0;
```

```
assign pp12 = (booth_bits[13:12] == 2'b01) ? ($signed(multiplicand) <<< 12) :
```

```
(booth_bits[13:12] == 2'b10) ? -($signed(multiplicand) <<< 12) : 32'sd0;
```

```
assign pp13 = (booth_bits[14:13] == 2'b01) ? ($signed(multiplicand) <<< 13) :
```

```
(booth_bits[14:13] == 2'b10) ? -($signed(multiplicand) <<< 13) : 32'sd0;
```

```
assign pp14 = (booth_bits[15:14] == 2'b01) ? ($signed(multiplicand) <<< 14) :
```

```
(booth_bits[15:14] == 2'b10) ? -($signed(multiplicand) <<< 14) : 32'sd0;
```

```
assign pp15 = (booth_bits[16:15] == 2'b01) ? ($signed(multiplicand) <<< 15) :
```

```
(booth_bits[16:15] == 2'b10) ? -($signed(multiplicand) <<< 15) : 32'sd0;
```

```
assign product =
```

```
    pp0 + pp1 + pp2 + pp3
```

```
    + pp4 + pp5 + pp6 + pp7
```

```
    + pp8 + pp9 + pp10 + pp11
```

```
    + pp12 + pp13 + pp14 + pp15;
```

endmodule

Prompt Strategy: Constraints First

Prompt:

Design Constraints:

1. Dataflow RTL only.
2. Synthesizable on Xilinx Artix-7 xc7a100tcsg324-2.
3. No always blocks.
4. No procedural logic.
5. Continuous assignments required.
6. Booth encoding must be implemented.
7. Partial products must be explicitly generated.
8. Signed 16-bit multiplication.
9. Output width must be 32 bits.
10. Combinational implementation only.

Task:

Generate a Radix-2 Booth Multiplier satisfying all constraints above.

output Verilog code only.

Code:

```
module booth_multiplier_16 (  
    input signed [15:0] multiplicand,  
    input signed [15:0] multiplier,  
    output signed [31:0] product  
);
```

```
wire [16:0] y;
```

```
assign y = {multiplier, 1'b0};
```

```
wire signed [31:0] pp0;
```

```
wire signed [31:0] pp1;
```

```
wire signed [31:0] pp2;
```

```
wire signed [31:0] pp3;
```

```
wire signed [31:0] pp4;
```

```
wire signed [31:0] pp5;
```

```
wire signed [31:0] pp6;
```

```
wire signed [31:0] pp7;
```

```
wire signed [31:0] pp8;
```

```
wire signed [31:0] pp9;
```

```
wire signed [31:0] pp10;
```

```
wire signed [31:0] pp11;
```

```
wire signed [31:0] pp12;
```

```
wire signed [31:0] pp13;
```

```
wire signed [31:0] pp14;
```

```
wire signed [31:0] pp15;
```

```
assign pp0 = (y[1:0] == 2'b01) ? ($signed(multiplicand) <<< 0 ) :  
    (y[1:0] == 2'b10) ? -($signed(multiplicand) <<< 0 ) :  
    32'sd0;
```

```
assign pp1 = (y[2:1] == 2'b01) ? ($signed(multiplicand) <<< 1 ) :  
    (y[2:1] == 2'b10) ? -($signed(multiplicand) <<< 1 ) :  
    32'sd0;
```

```
assign pp2 = (y[3:2] == 2'b01) ? ($signed(multiplicand) <<< 2 ) :  
    (y[3:2] == 2'b10) ? -($signed(multiplicand) <<< 2 ) :  
    32'sd0;
```

```
assign pp3 = (y[4:3] == 2'b01) ? ($signed(multiplicand) <<< 3 ) :  
    (y[4:3] == 2'b10) ? -($signed(multiplicand) <<< 3 ) :  
    32'sd0;
```

```
assign pp4 = (y[5:4] == 2'b01) ? ($signed(multiplicand) <<< 4 ) :  
    (y[5:4] == 2'b10) ? -($signed(multiplicand) <<< 4 ) :  
    32'sd0;
```

```
assign pp5 = (y[6:5] == 2'b01) ? ($signed(multiplicand) <<< 5 ) :  
    (y[6:5] == 2'b10) ? -($signed(multiplicand) <<< 5 ) :  
    32'sd0;
```

```
assign pp6 = (y[7:6] == 2'b01) ? ($signed(multiplicand) <<< 6 ) :
```

```
(y[7:6] == 2'b10) ? -($signed(multiplicand) <<< 6) :  
32'sd0;
```

```
assign pp7 = (y[8:7] == 2'b01) ? ($signed(multiplicand) <<< 7) :  
(y[8:7] == 2'b10) ? -($signed(multiplicand) <<< 7) :  
32'sd0;
```

```
assign pp8 = (y[9:8] == 2'b01) ? ($signed(multiplicand) <<< 8) :  
(y[9:8] == 2'b10) ? -($signed(multiplicand) <<< 8) :  
32'sd0;
```

```
assign pp9 = (y[10:9] == 2'b01) ? ($signed(multiplicand) <<< 9) :  
(y[10:9] == 2'b10) ? -($signed(multiplicand) <<< 9) :  
32'sd0;
```

```
assign pp10 = (y[11:10] == 2'b01) ? ($signed(multiplicand) <<< 10) :  
(y[11:10] == 2'b10) ? -($signed(multiplicand) <<< 10) :  
32'sd0;
```

```
assign pp11 = (y[12:11] == 2'b01) ? ($signed(multiplicand) <<< 11) :  
(y[12:11] == 2'b10) ? -($signed(multiplicand) <<< 11) :  
32'sd0;
```

```
assign pp12 = (y[13:12] == 2'b01) ? ($signed(multiplicand) <<< 12) :  
(y[13:12] == 2'b10) ? -($signed(multiplicand) <<< 12) :  
32'sd0;
```

```
assign pp13 = (y[14:13] == 2'b01) ? ($signed(multiplicand) <<< 13) :
```

```
(y[14:13] == 2'b10) ? -($signed(multiplicand) <<< 13) :
```

```
32'sd0;
```

```
assign pp14 = (y[15:14] == 2'b01) ? ($signed(multiplicand) <<< 14) :
```

```
(y[15:14] == 2'b10) ? -($signed(multiplicand) <<< 14) :
```

```
32'sd0;
```

```
assign pp15 = (y[16:15] == 2'b01) ? ($signed(multiplicand) <<< 15) :
```

```
(y[16:15] == 2'b10) ? -($signed(multiplicand) <<< 15) :
```

```
32'sd0;
```

```
assign product =
```

```
    pp0 + pp1 + pp2 + pp3
```

```
    + pp4 + pp5 + pp6 + pp7
```

```
    + pp8 + pp9 + pp10 + pp11
```

```
    + pp12 + pp13 + pp14 + pp15;
```

```
endmodule
```

Prompt Strategy: Self Planning

Prompt:

Generate a 16-bit signed Booth Multiplier.

First create an implementation plan.

The plan should identify:

- Booth encoding strategy
- Partial-product generation strategy
- Dataflow expression structure
- Summation strategy
- Output generation method

After creating the plan, implement the design according to the plan.

Requirements:

- Radix-2 Booth multiplication.
- Dataflow RTL only.
- Synthesizable Verilog.
- No procedural blocks.
- No always statements.
- Target FPGA: xc7a100tcs9364-2.

Output complete code.

Code:

```
module booth_multiplier_16bit (  
    input signed [15:0] multiplicand,  
    input signed [15:0] multiplier,
```

```

output signed [31:0] product

);

// Sign-extended multiplicand
wire signed [31:0] m_ext;

// Extended multiplier for Booth encoding
wire [16:0] q_ext;

assign m_ext = {{16{multiplicand[15]}}, multiplicand};
assign q_ext = {multiplier, 1'b0};

// Partial products
wire signed [31:0] pp0;
wire signed [31:0] pp1;
wire signed [31:0] pp2;
wire signed [31:0] pp3;
wire signed [31:0] pp4;
wire signed [31:0] pp5;
wire signed [31:0] pp6;
wire signed [31:0] pp7;
wire signed [31:0] pp8;
wire signed [31:0] pp9;
wire signed [31:0] pp10;
wire signed [31:0] pp11;
wire signed [31:0] pp12;

```

wire signed [31:0] pp13;

wire signed [31:0] pp14;

wire signed [31:0] pp15;

```
assign pp0 = (q_ext[1:0] == 2'b01) ? (m_ext <<< 0) :  
             (q_ext[1:0] == 2'b10) ? -(m_ext <<< 0) :  
             32'sd0;
```

```
assign pp1 = (q_ext[2:1] == 2'b01) ? (m_ext <<< 1) :  
             (q_ext[2:1] == 2'b10) ? -(m_ext <<< 1) :  
             32'sd0;
```

```
assign pp2 = (q_ext[3:2] == 2'b01) ? (m_ext <<< 2) :  
             (q_ext[3:2] == 2'b10) ? -(m_ext <<< 2) :  
             32'sd0;
```

```
assign pp3 = (q_ext[4:3] == 2'b01) ? (m_ext <<< 3) :  
             (q_ext[4:3] == 2'b10) ? -(m_ext <<< 3) :  
             32'sd0;
```

```
assign pp4 = (q_ext[5:4] == 2'b01) ? (m_ext <<< 4) :  
             (q_ext[5:4] == 2'b10) ? -(m_ext <<< 4) :  
             32'sd0;
```

```
assign pp5 = (q_ext[6:5] == 2'b01) ? (m_ext <<< 5) :  
             (q_ext[6:5] == 2'b10) ? -(m_ext <<< 5) :
```

```
32'sd0;
```

```
assign pp6 = (q_ext[7:6] == 2'b01) ? (m_ext <<< 6) :
```

```
(q_ext[7:6] == 2'b10) ? -(m_ext <<< 6) :
```

```
32'sd0;
```

```
assign pp7 = (q_ext[8:7] == 2'b01) ? (m_ext <<< 7) :
```

```
(q_ext[8:7] == 2'b10) ? -(m_ext <<< 7) :
```

```
32'sd0;
```

```
assign pp8 = (q_ext[9:8] == 2'b01) ? (m_ext <<< 8) :
```

```
(q_ext[9:8] == 2'b10) ? -(m_ext <<< 8) :
```

```
32'sd0;
```

```
assign pp9 = (q_ext[10:9] == 2'b01) ? (m_ext <<< 9) :
```

```
(q_ext[10:9] == 2'b10) ? -(m_ext <<< 9) :
```

```
32'sd0;
```

```
assign pp10 = (q_ext[11:10] == 2'b01) ? (m_ext <<< 10) :
```

```
(q_ext[11:10] == 2'b10) ? -(m_ext <<< 10) :
```

```
32'sd0;
```

```
assign pp11 = (q_ext[12:11] == 2'b01) ? (m_ext <<< 11) :
```

```
(q_ext[12:11] == 2'b10) ? -(m_ext <<< 11) :
```

```
32'sd0;
```

```
assign pp12 = (q_ext[13:12] == 2'b01) ? (m_ext <<< 12) :  
    (q_ext[13:12] == 2'b10) ? -(m_ext <<< 12) :  
    32'sd0;
```

```
assign pp13 = (q_ext[14:13] == 2'b01) ? (m_ext <<< 13) :  
    (q_ext[14:13] == 2'b10) ? -(m_ext <<< 13) :  
    32'sd0;
```

```
assign pp14 = (q_ext[15:14] == 2'b01) ? (m_ext <<< 14) :  
    (q_ext[15:14] == 2'b10) ? -(m_ext <<< 14) :  
    32'sd0;
```

```
assign pp15 = (q_ext[16:15] == 2'b01) ? (m_ext <<< 15) :  
    (q_ext[16:15] == 2'b10) ? -(m_ext <<< 15) :  
    32'sd0;
```

```
// Reduction tree
```

```
wire signed [31:0] s0;
```

```
wire signed [31:0] s1;
```

```
wire signed [31:0] s2;
```

```
wire signed [31:0] s3;
```

```
wire signed [31:0] s4;
```

```
wire signed [31:0] s5;
```

```
wire signed [31:0] s6;
```

```
wire signed [31:0] s7;
```

```
assign s0 = pp0 + pp1;  
assign s1 = pp2 + pp3;  
assign s2 = pp4 + pp5;  
assign s3 = pp6 + pp7;  
assign s4 = pp8 + pp9;  
assign s5 = pp10 + pp11;  
assign s6 = pp12 + pp13;  
assign s7 = pp14 + pp15;
```

```
wire signed [31:0] t0;  
wire signed [31:0] t1;  
wire signed [31:0] t2;  
wire signed [31:0] t3;
```

```
assign t0 = s0 + s1;  
assign t1 = s2 + s3;  
assign t2 = s4 + s5;  
assign t3 = s6 + s7;
```

```
wire signed [31:0] u0;  
wire signed [31:0] u1;
```

```
assign u0 = t0 + t1;  
assign u1 = t2 + t3;
```

```
wire signed [31:0] final_sum;
```

```
assign final_sum = u0 + u1;
```

```
assign product = final_sum;
```

```
endmodule
```

Prompt Strategy: Iterative Correction

Prompt:

Generate synthesizable Verilog for a 16-bit signed Booth Multiplier.

After generating the initial design:

1. Review the RTL for violations of Dataflow RTL style.
2. Remove procedural constructs if present.
3. Verify continuous assignments are used.
4. Verify Booth encoding implementation.
5. Verify partial-product generation.
6. Verify synthesizability.
7. Correct any identified issues.

Then provide the final corrected version.

Requirements:

- Radix-2 Booth multiplication.

- Dataflow RTL only.
- Target FPGA: xc7a100tcsg324-2.
- Output final code only.

Code:

```

module booth_multiplier_16bit (
    input signed [15:0] multiplicand,
    input signed [15:0] multiplier,
    output signed [31:0] product
);

    // Booth bit-pair evaluation

    wire [16:0] booth_bits;

    assign booth_bits = {multiplier, 1'b0};

    // Sign-extended multiplicand

    wire signed [31:0] mcand_ext;

    assign mcand_ext = {{16{multiplicand[15]}}, multiplicand};

    // Partial products

    wire signed [31:0] pp0;

    wire signed [31:0] pp1;

    wire signed [31:0] pp2;

    wire signed [31:0] pp3;

    wire signed [31:0] pp4;

```



```
wire signed [31:0] pp5;  
wire signed [31:0] pp6;  
wire signed [31:0] pp7;  
wire signed [31:0] pp8;  
wire signed [31:0] pp9;  
wire signed [31:0] pp10;  
wire signed [31:0] pp11;  
wire signed [31:0] pp12;  
wire signed [31:0] pp13;  
wire signed [31:0] pp14;  
wire signed [31:0] pp15;
```

```
assign pp0 = (booth_bits[1:0] == 2'b01) ? (mcand_ext <<< 0) :  
              (booth_bits[1:0] == 2'b10) ? -(mcand_ext <<< 0) :  
              32'sd0;
```

```
assign pp1 = (booth_bits[2:1] == 2'b01) ? (mcand_ext <<< 1) :  
              (booth_bits[2:1] == 2'b10) ? -(mcand_ext <<< 1) :  
              32'sd0;
```

```
assign pp2 = (booth_bits[3:2] == 2'b01) ? (mcand_ext <<< 2) :  
              (booth_bits[3:2] == 2'b10) ? -(mcand_ext <<< 2) :  
              32'sd0;
```

```
assign pp3 = (booth_bits[4:3] == 2'b01) ? (mcand_ext <<< 3) :  
              (booth_bits[4:3] == 2'b10) ? -(mcand_ext <<< 3) :
```

32'sd0;

assign pp4 = (booth_bits[5:4] == 2'b01) ? (mcand_ext <<< 4) :

(booth_bits[5:4] == 2'b10) ? -(mcand_ext <<< 4) :

32'sd0;

assign pp5 = (booth_bits[6:5] == 2'b01) ? (mcand_ext <<< 5) :

(booth_bits[6:5] == 2'b10) ? -(mcand_ext <<< 5) :

32'sd0;

assign pp6 = (booth_bits[7:6] == 2'b01) ? (mcand_ext <<< 6) :

(booth_bits[7:6] == 2'b10) ? -(mcand_ext <<< 6) :

32'sd0;

assign pp7 = (booth_bits[8:7] == 2'b01) ? (mcand_ext <<< 7) :

(booth_bits[8:7] == 2'b10) ? -(mcand_ext <<< 7) :

32'sd0;

assign pp8 = (booth_bits[9:8] == 2'b01) ? (mcand_ext <<< 8) :

(booth_bits[9:8] == 2'b10) ? -(mcand_ext <<< 8) :

32'sd0;

assign pp9 = (booth_bits[10:9] == 2'b01) ? (mcand_ext <<< 9) :

(booth_bits[10:9] == 2'b10) ? -(mcand_ext <<< 9) :

32'sd0;

```
assign pp10 = (booth_bits[11:10] == 2'b01) ? (mcand_ext <<< 10) :  
    (booth_bits[11:10] == 2'b10) ? -(mcand_ext <<< 10) :  
    32'sd0;
```

```
assign pp11 = (booth_bits[12:11] == 2'b01) ? (mcand_ext <<< 11) :  
    (booth_bits[12:11] == 2'b10) ? -(mcand_ext <<< 11) :  
    32'sd0;
```

```
assign pp12 = (booth_bits[13:12] == 2'b01) ? (mcand_ext <<< 12) :  
    (booth_bits[13:12] == 2'b10) ? -(mcand_ext <<< 12) :  
    32'sd0;
```

```
assign pp13 = (booth_bits[14:13] == 2'b01) ? (mcand_ext <<< 13) :  
    (booth_bits[14:13] == 2'b10) ? -(mcand_ext <<< 13) :  
    32'sd0;
```

```
assign pp14 = (booth_bits[15:14] == 2'b01) ? (mcand_ext <<< 14) :  
    (booth_bits[15:14] == 2'b10) ? -(mcand_ext <<< 14) :  
    32'sd0;
```

```
assign pp15 = (booth_bits[16:15] == 2'b01) ? (mcand_ext <<< 15) :  
    (booth_bits[16:15] == 2'b10) ? -(mcand_ext <<< 15) :  
    32'sd0;
```

```
// Partial-product summation
```

```
assign product =
```

```
        pp0 + pp1 + pp2 + pp3
      + pp4 + pp5 + pp6 + pp7
      + pp8 + pp9 + pp10 + pp11
      + pp12 + pp13 + pp14 + pp15;

endmodule
```

Prompt Strategy: Hybrid

Prompt:

You are a Senior FPGA Design Engineer.

TASK:

Generate synthesizable Verilog for a 16-bit signed Booth Multiplier.

Before coding:

1. Analyze the Booth algorithm.
2. Create an implementation plan.
3. Determine Booth encoding strategy.
4. Determine partial-product generation strategy.
5. Determine summation strategy.

CONSTRAINTS:

- Dataflow RTL only.
- No always blocks.
- No procedural constructs.

- Continuous assignments preferred.
- Combinational implementation only.
- Target FPGA: Xilinx Artix-7 xc7a100tcsg324-2.

SELF-CHECK:

- Verify Dataflow compliance.
- Verify Booth encoding implementation.
- Verify synthesizability.
- Verify correct port widths.

OUTPUT:

Verilog code only.

Code:

```
module booth_multiplier_16bit (
    input signed [15:0] multiplicand,
    input signed [15:0] multiplier,
    output signed [31:0] product
);

    // Extended multiplier for Booth encoding:
    // {multiplier[15], multiplier[15:0], 1'b0}
    wire [16:0] booth_bits;

    assign booth_bits = {multiplier, 1'b0};
```

```
// Booth partial products
```

```
wire signed [31:0] pp0;
```

```
wire signed [31:0] pp1;
```

```
wire signed [31:0] pp2;
```

```
wire signed [31:0] pp3;
```

```
wire signed [31:0] pp4;
```

```
wire signed [31:0] pp5;
```

```
wire signed [31:0] pp6;
```

```
wire signed [31:0] pp7;
```

```
wire signed [31:0] pp8;
```

```
wire signed [31:0] pp9;
```

```
wire signed [31:0] pp10;
```

```
wire signed [31:0] pp11;
```

```
wire signed [31:0] pp12;
```

```
wire signed [31:0] pp13;
```

```
wire signed [31:0] pp14;
```

```
wire signed [31:0] pp15;
```

```
// Radix-2 Booth encoding
```

```
// 01 -> +M
```

```
// 10 -> -M
```

```
// 00,11 -> 0
```

```
assign pp0 = (booth_bits[1:0] == 2'b01) ? ($signed({16{multiplicand[15]}}, multiplicand))  
<<< 0 ) :
```

```
        (booth_bits[1:0] == 2'b10) ? (~$signed({16{multiplicand[15]}}, multiplicand))
<<< 0 )) :
        32'sd0;
```

```
    assign pp1 = (booth_bits[2:1] == 2'b01) ? ($signed({16{multiplicand[15]}}, multiplicand))
<<< 1 )) :
        (booth_bits[2:1] == 2'b10) ? (~$signed({16{multiplicand[15]}}, multiplicand))
<<< 1 )) :
        32'sd0;
```

```
    assign pp2 = (booth_bits[3:2] == 2'b01) ? ($signed({16{multiplicand[15]}}, multiplicand))
<<< 2 )) :
        (booth_bits[3:2] == 2'b10) ? (~$signed({16{multiplicand[15]}}, multiplicand))
<<< 2 )) :
        32'sd0;
```

```
    assign pp3 = (booth_bits[4:3] == 2'b01) ? ($signed({16{multiplicand[15]}}, multiplicand))
<<< 3 )) :
        (booth_bits[4:3] == 2'b10) ? (~$signed({16{multiplicand[15]}}, multiplicand))
<<< 3 )) :
        32'sd0;
```

```
    assign pp4 = (booth_bits[5:4] == 2'b01) ? ($signed({16{multiplicand[15]}}, multiplicand))
<<< 4 )) :
        (booth_bits[5:4] == 2'b10) ? (~$signed({16{multiplicand[15]}}, multiplicand))
<<< 4 )) :
        32'sd0;
```

```
    assign pp5 = (booth_bits[6:5] == 2'b01) ? ($signed({16{multiplicand[15]}}, multiplicand))
<<< 5 )) :
```

```
        (booth_bits[6:5] == 2'b10) ? (~$signed({16{multiplicand[15]}}, multiplicand))
<<< 5 )) :
        32'sd0;
```

```
    assign pp6 = (booth_bits[7:6] == 2'b01) ? ($signed({16{multiplicand[15]}}, multiplicand))
<<< 6 )) :
```

```
        (booth_bits[7:6] == 2'b10) ? (~$signed({16{multiplicand[15]}}, multiplicand))
<<< 6 )) :
        32'sd0;
```

```
    assign pp7 = (booth_bits[8:7] == 2'b01) ? ($signed({16{multiplicand[15]}}, multiplicand))
<<< 7 )) :
```

```
        (booth_bits[8:7] == 2'b10) ? (~$signed({16{multiplicand[15]}}, multiplicand))
<<< 7 )) :
        32'sd0;
```

```
    assign pp8 = (booth_bits[9:8] == 2'b01) ? ($signed({16{multiplicand[15]}}, multiplicand))
<<< 8 )) :
```

```
        (booth_bits[9:8] == 2'b10) ? (~$signed({16{multiplicand[15]}}, multiplicand))
<<< 8 )) :
        32'sd0;
```

```
    assign pp9 = (booth_bits[10:9] == 2'b01) ? ($signed({16{multiplicand[15]}},
multiplicand)) <<< 9 )) :
```

```
        (booth_bits[10:9] == 2'b10) ? (~$signed({16{multiplicand[15]}}, multiplicand))
<<< 9 )) :
        32'sd0;
```

```
    assign pp10 = (booth_bits[11:10] == 2'b01) ? ($signed({16{multiplicand[15]}},
multiplicand)) <<< 10 )) :
```



```
(booth_bits[11:10] == 2'b10) ? (-($signed({16{multiplicand[15]}}, multiplicand))  
<<< 10)) :
```

```
32'sd0;
```

```
assign pp11 = (booth_bits[12:11] == 2'b01) ? ($signed({16{multiplicand[15]}},  
multiplicand)) <<< 11) :
```

```
(booth_bits[12:11] == 2'b10) ? (-($signed({16{multiplicand[15]}}, multiplicand))  
<<< 11)) :
```

```
32'sd0;
```

```
assign pp12 = (booth_bits[13:12] == 2'b01) ? ($signed({16{multiplicand[15]}},  
multiplicand)) <<< 12) :
```

```
(booth_bits[13:12] == 2'b10) ? (-($signed({16{multiplicand[15]}}, multiplicand))  
<<< 12)) :
```

```
32'sd0;
```

```
assign pp13 = (booth_bits[14:13] == 2'b01) ? ($signed({16{multiplicand[15]}},  
multiplicand)) <<< 13) :
```

```
(booth_bits[14:13] == 2'b10) ? (-($signed({16{multiplicand[15]}}, multiplicand))  
<<< 13)) :
```

```
32'sd0;
```

```
assign pp14 = (booth_bits[15:14] == 2'b01) ? ($signed({16{multiplicand[15]}},  
multiplicand)) <<< 14) :
```

```
(booth_bits[15:14] == 2'b10) ? (-($signed({16{multiplicand[15]}}, multiplicand))  
<<< 14)) :
```

```
32'sd0;
```

```
assign pp15 = (booth_bits[16:15] == 2'b01) ? ($signed({16{multiplicand[15]}},  
multiplicand)) <<< 15) :
```

```
        (booth_bits[16:15] == 2'b10) ? (-($signed({16{multiplicand[15]}}, multiplicand)
<<< 15)) :
```

```
        32'sd0;
```

```
// Partial-product summation
```

```
assign product =
```

```
    pp0 + pp1 + pp2 + pp3
```

```
    + pp4 + pp5 + pp6 + pp7
```

```
    + pp8 + pp9 + pp10 + pp11
```

```
    + pp12 + pp13 + pp14 + pp15;
```

```
endmodule
```